

230905290 PPL LAB 10

Rohan Shenoy Roll no 32 CSE D1

Q1] Write a program in CUDA to perform matrix multiplication using 2D Grid and 2D Block.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <device_launch_parameters.h>
#define MAX 100
__global__ void matrixMul(int *A, int *B, int *C, int n, int m, int p) {

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < n && col < p) {
        int sum = 0;
        for (int k = 0; k < m; k++) {
            sum += A[row * m + k] * B[k * p + col];
        }
        C[row * p + col] = sum;
    }
}

int main() {
    int A[MAX], B[MAX], C[MAX];
    int n, m, p;
    printf("Enter rows and cols of Matrix A: ");
    scanf("%d %d", &n, &m);
    printf("Enter cols of Matrix B: ");
    scanf("%d", &p);
    printf("Enter Matrix A:\n");
    for (int i = 0; i < n * m; i++)
        scanf("%d", &A[i]);
    printf("Enter Matrix B:\n");
    for (int i = 0; i < m * p; i++)
```

```

    scanf("%d", &B[i]);
int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, n * m * sizeof(int));
cudaMalloc(&d_B, m * p * sizeof(int));
cudaMalloc(&d_C, n * p * sizeof(int));
cudaMemcpy(d_A, A, n * m * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B, m * p * sizeof(int), cudaMemcpyHostToDevice);
dim3 block(2, 2);
dim3 grid((p + 1) / 2, (n + 1) / 2);
matrixMul<<<grid, block>>>(d_A, d_B, d_C, n, m, p);
cudaMemcpy(C, d_C, n * p * sizeof(int), cudaMemcpyDeviceToHost);
printf("Result Matrix:\n");
for (int i = 0; i < n * p; i++) {
printf("%d ", C[i]);
if ((i + 1) % p == 0) printf("\n");
}
return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/pp1_lab/lab10$ ./a.out
Enter rows and cols of Matrix A: 2 2
Enter cols of Matrix B: 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
5 6
7 8
Result Matrix:
19 22
43 50

```

Q2] Write a program in CUDA to improve the performance of 1D parallel convolution using constant Memory.

CODE]

```

#include <stdio.h>

```

```

#include <cuda_runtime.h>
#include <stdlib.h>
#include <device_launch_parameters.h>

#define MAX 100
__constant__ int d_mask[MAX];
__global__ void convolution(int *N, int *P, int width, int mask_width) {
    int i = threadIdx.x;
    int sum = 0;
    int half = mask_width / 2;
    if (i < width) {
        for (int j = 0; j < mask_width; j++) {
            int idx = i - half + j;
            if (idx >= 0 && idx < width)
                sum += N[idx] * d_mask[j];
        }
        P[i] = sum;
    }
}

int main()
{
    int N[MAX], M[MAX], P[MAX];
    int width, mask_width;
    printf("Enter input size: ");
    scanf("%d", &width);
    printf("Enter input array:\n");
    for (int i = 0; i < width; i++)
        scanf("%d", &N[i]);
    printf("Enter mask size: ");
    scanf("%d", &mask_width);
    printf("Enter mask array:\n");
    for (int i = 0; i < mask_width; i++)
        scanf("%d", &M[i]);
    int *d_N, *d_P;
    cudaMalloc(&d_N, width * sizeof(int));
    cudaMalloc(&d_P, width * sizeof(int));
    cudaMemcpy(d_N, N, width * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpyToSymbol(d_mask, M, mask_width * sizeof(int));
    convolution<<<1, width>>>(d_N, d_P, width, mask_width);
}

```

```

cudaMemcpy(P, d_P, width * sizeof(int), cudaMemcpyDeviceToHost);

printf("Result:\n");
for (int i = 0; i < width; i++)
    printf("%d ", P[i]);
printf("\n");
return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/pp1_lab/lab10$ nvcc q2.cu
6CSED1@debian:~/Desktop/pp1_lab/lab10$ ./a.out
Enter input size: 5
Enter input array:
1 2 3 4 5
Enter mask size: 3
Enter mask array:
1 0 -1
Result:
-2 -2 -2 -2 4
6CSED1@debian:~/Desktop/pp1_lab/lab10$ 

```

Q3] Write a program in CUDA to perform tiled 1D convolution operation on the input array N of size width using the mask array, M of size mask_width, to produce the resultant array P of size width

CODE]

```

#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <device_launch_parameters.h>
#define MAX 100
__global__ void tiledConv(int *N, int *M, int *P, int width, int
mask_width) {
    __shared__ int tile[MAX];
    int tx = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tx;
    int half = mask_width / 2;
    tile[tx] = (i < width) ? N[i] : 0;
}

```

```

__syncthreads();
int sum = 0;
if (i < width) {
    for (int j = 0; j < mask_width; j++) {
        int idx = tx - half + j;
        if (idx >= 0 && idx < blockDim.x)
            sum += tile[idx] * M[j];
    }
    P[i] = sum;
}
}

int main()
{
    int N[MAX], M[MAX], P[MAX];
    int width, mask_width;
    printf("Enter input size: ");
    scanf("%d", &width);
    printf("Enter input array:\n");
    for (int i = 0; i < width; i++)
        scanf("%d", &N[i]);
    printf("Enter mask size: ");
    scanf("%d", &mask_width);
    printf("Enter mask array:\n");
    for (int i = 0; i < mask_width; i++)
        scanf("%d", &M[i]);
    int *d_N, *d_M, *d_P;
    cudaMalloc(&d_N, width * sizeof(int));
    cudaMalloc(&d_M, mask_width * sizeof(int));
    cudaMalloc(&d_P, width * sizeof(int));
    cudaMemcpy(d_N, N, width * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_M, M, mask_width * sizeof(int), cudaMemcpyHostToDevice);
    tiledConv<<<1, width>>>(d_N, d_M, d_P, width, mask_width);
    cudaMemcpy(P, d_P, width * sizeof(int), cudaMemcpyDeviceToHost);

    printf("Result:\n");
    for (int i = 0; i < width; i++)
        printf("%d ", P[i]);
    printf("\n");
}

```

```
return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/pp1_lab/lab10$ ./a.out
Enter input size: 5
Enter input array:
1
2
3
4
5
Enter mask size: 3
Enter mask array:
1 1 1
Result:
3 6 9 12 9
```

ADDITIONAL COURSE:

Q1] Write a program in CUDA which displays a shopping mall item menu with its price. The N number of friends are allowed to purchase as many items they want. Calculate the total purchase done by N friends.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <device_launch_parameters.h>
#define MAX 100
__global__ void totalCost(int *prices, int *quantities, int *total, int n)
{
    int i = threadIdx.x;
    if (i < n) {
        total[i] = prices[i] * quantities[i];
    }
}
int main() {
```

```

int prices[MAX], quantities[MAX], total[MAX];
int n;
printf("Enter number of items: ");
scanf("%d", &n);
printf("Enter prices:\n");
for (int i = 0; i < n; i++)
scanf("%d", &prices[i]);
printf("Enter quantities bought:\n");
for (int i = 0; i < n; i++)
scanf("%d", &quantities[i]);
int *d_prices, *d_quantities, *d_total;
cudaMalloc(&d_prices, n * sizeof(int));
cudaMalloc(&d_quantities, n * sizeof(int));
cudaMalloc(&d_total, n * sizeof(int));
cudaMemcpy(d_prices, prices, n * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_quantities, quantities, n * sizeof(int),
cudaMemcpyHostToDevice);
totalCost<<<1, n>>>(d_prices, d_quantities, d_total, n);
cudaMemcpy(total, d_total, n * sizeof(int), cudaMemcpyDeviceToHost);
int grandTotal = 0;
printf("Individual totals:\n");
for (int i = 0; i < n; i++) {
printf("%d ", total[i]);
grandTotal += total[i];
}
printf("\nGrand Total: %d\n", grandTotal);
return 0;
}

```

OUTPUT]

```
6CSED1@debian:~/Desktop/pp1_lab/lab10$ nvcc aq1.cu
6CSED1@debian:~/Desktop/pp1_lab/lab10$ ./a.out
Enter number of items: 3
Enter prices:
10 20 30
Enter quantities bought:
1 2 3
Individual totals:
10 40 90
Grand Total: 140
6CSED1@debian:~/Desktop/pp1_lab/lab10$
```